



Project and Instrumentation Design

2019/2020

Pulsemeter

Hugo Sebastião

Ferdinando Catania

INDEX

1. System development and circuit analysis
 - 1.1 Scheme of the circuit
 - 1.2 Breadboard
 - 1.3 KY-039
 - 1.4 AD-8232
 - 1.5 LCD display 1602
 - 1.6 Real circuit
2. Communication
3. Software development
 - 3.1 Flowchart matlab
 - 3.2 Data acquisition
 - 3.3 Coding and debugging
 - 3.3.1 Code and processing arduino
 - 3.3.2 Code and processing matlab
4. Demo
 - 4.1 Demo voltage drop
 - 4.2 Demo sensor KY-039 and display contrast
5. Conclusions

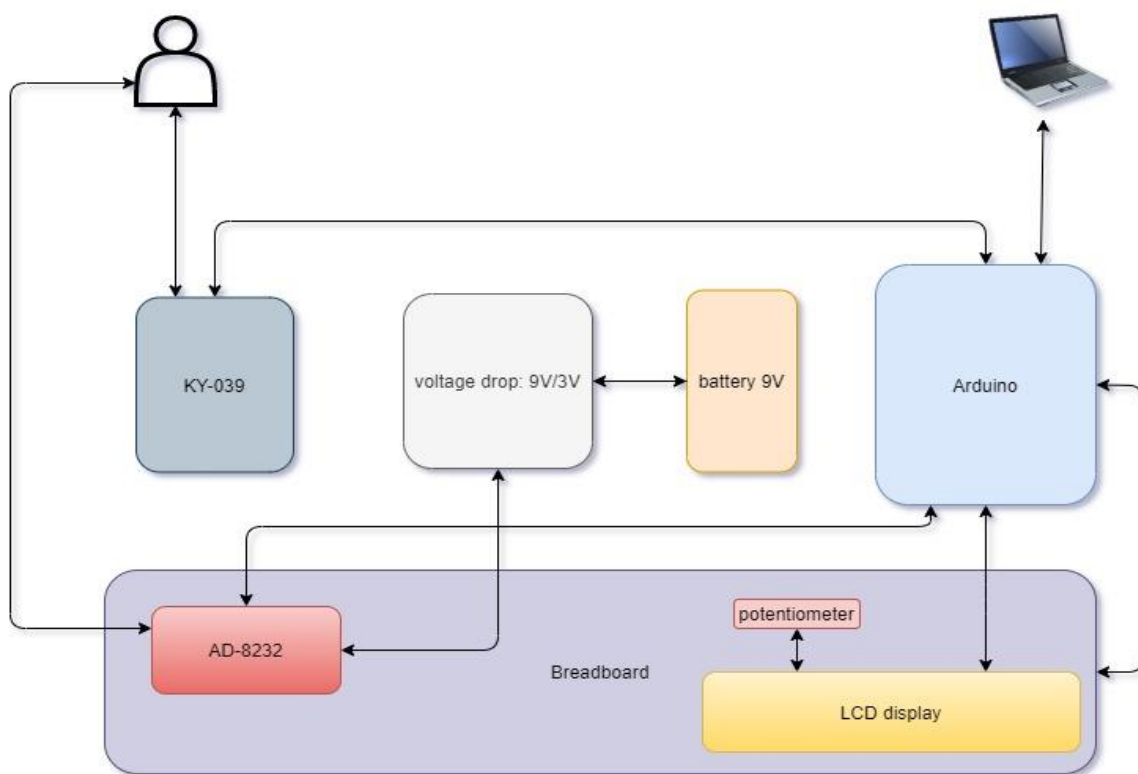
1. System development , circuit analysis and connections

In this part we are going to talk about the electric circuit and how we developed it, explaining all our reasoning throughout the design phase.

The main goal of this project is to analyse the heart rate with a sensor compatible with arduino. Searching on the internet allowed us to see the range of sensors suitable for this purpose and to make the project more profound we decided to buy the cheapest one and we tried to make it work correctly. Subsequently, we compared its results with heart rate from ECG.

Scheme of the circuit

In the figure below are shown all the main components that we used and how we linked them together.



reference diagram: <https://www.draw.io/>

Breadboard

The breadboard is used as a power supply with 5V taking current from arduino. We also used it to place the AD-8232 kit, LCD display and the potentiometer.

KY-039

The first approach was to test the sensor *KY-039* evaluating its reliability compared with the heart rate system of an apple watch 3.0. We immediately noticed their differences in terms of accuracy but we made sure the sensor worked well testing it in rest and high activity moments.

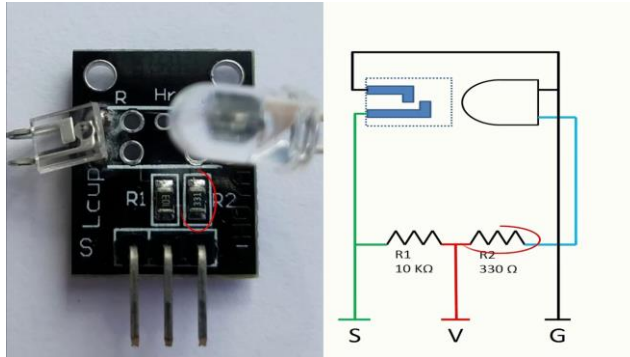


Fig. 1: KY-039 sensor with circuit diagram

This sensor has 3 pins, that are: ground(G), voltage(V) and analog output(S).

The voltage and ground pin have been linked with the breadboard while the analog output was connected with analog in (A1) of arduino.

AD-8232

The second step was to link the *AD-8232 kit for ECG*. This kit has already internal components which filter and amplify the signal. Nevertheless, linking the kit to arduino power supply, we noticed the influence of 50 Hz noise overrides the useful signal.

Instead of removing this noise filtering the signal by arduino or matlab we thought of another method that seems to work very well: it consists of using as a power supply a DC battery of 9V (battery we had available with arduino kit) for voltage supply instead of the arduino uno.

The only problem was the kit AD-8232 needs 3.3V and not 9V, to overcome this problem we built a voltage divider using two resistors respectively of 860kΩ and 500kΩ. The output of the ECG is linked at analog pin (A1) of arduino.

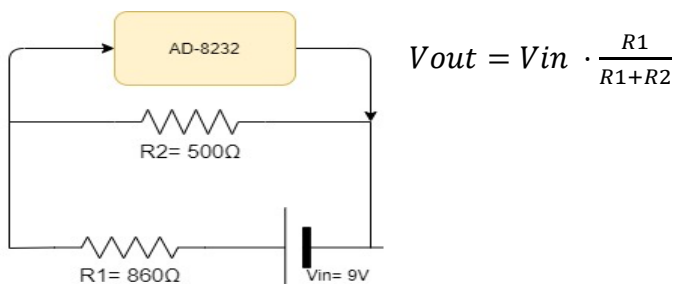


Fig. 2: power divider (9V->3.3V)

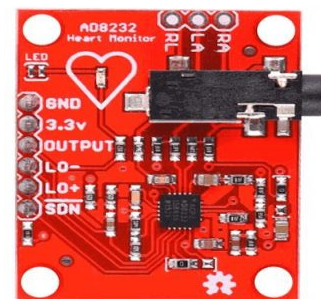


Fig. 3: AD-8232 kit

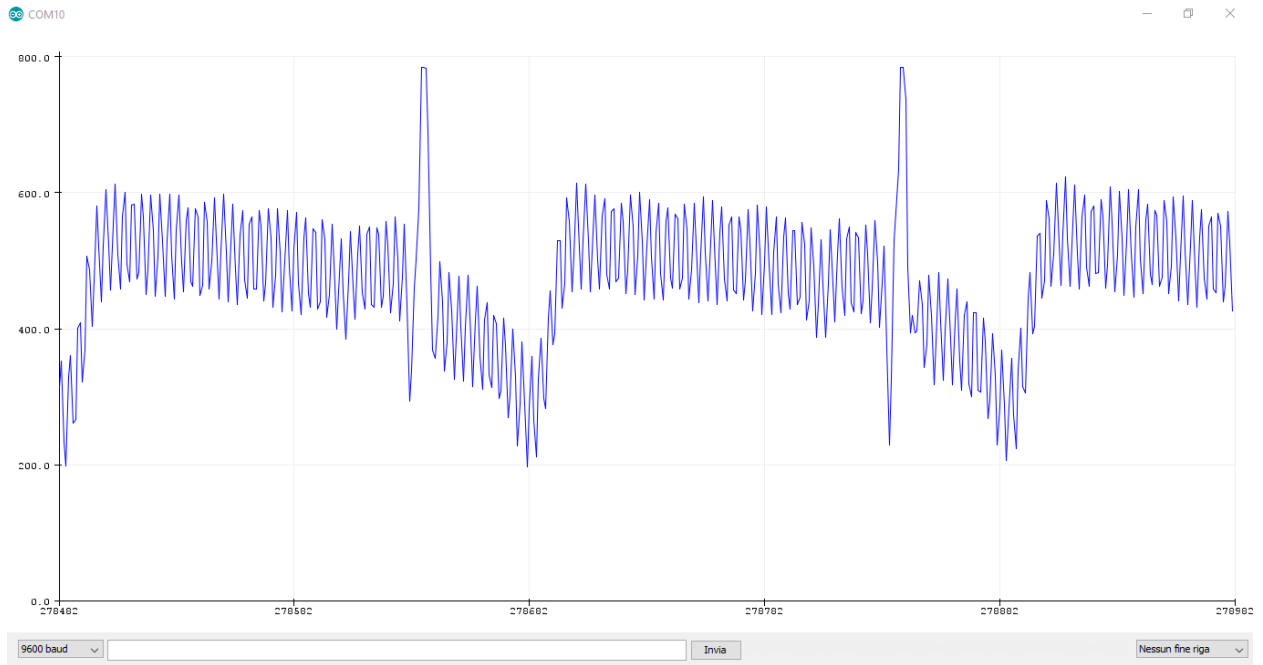


Fig. 4: ECG signal with power supply of arduino

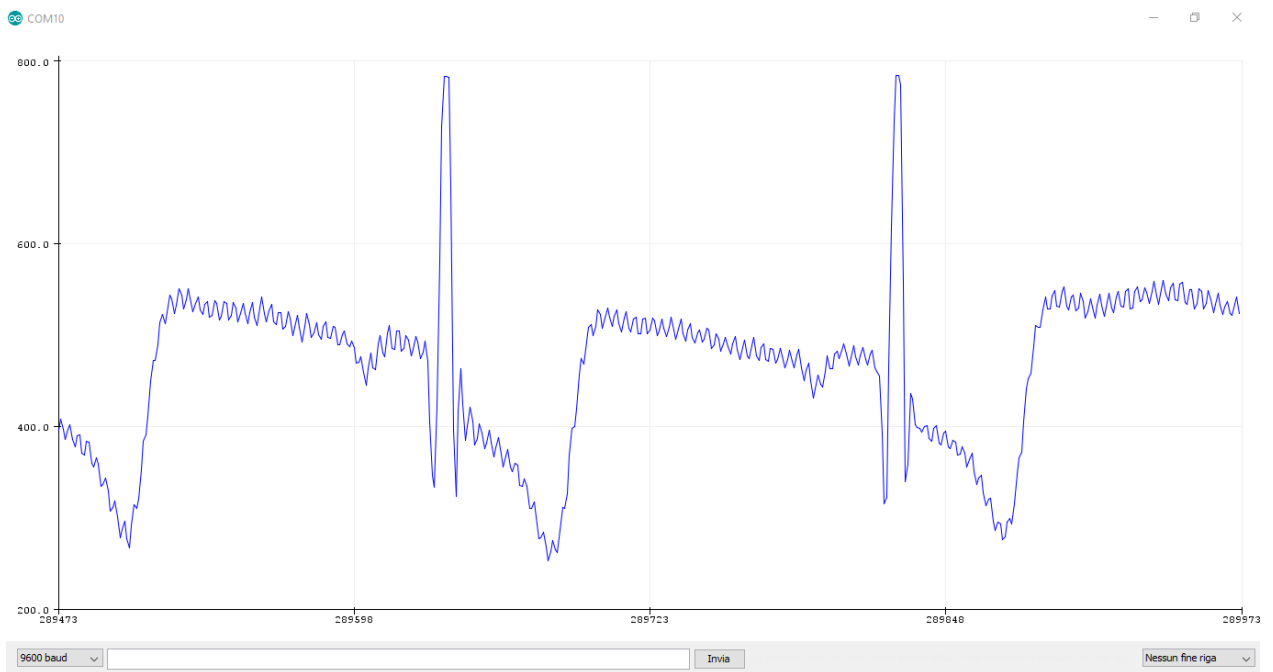


Fig. 5: ECG signal with power supply of 3.3V DC

LCD display 1602

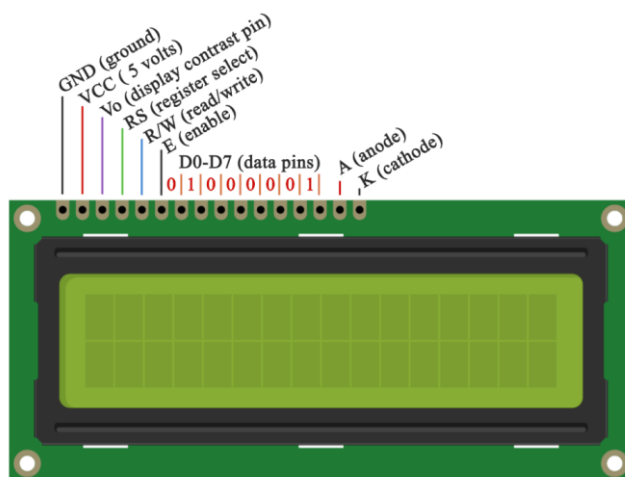
To make the sensor more realistic for its use, we added an external display which allowed us to show the HR without the use of a computer; in fact, arduino saves the program

in its internal memory and runs it until there is power supply (for electrical portable device we always needed a battery).

Introduction to the pins of *LCD1602*:

- VSS: A pin that connects to ground
- VDD: A pin that connects to a 5V power supply
- VO: A pin that adjusts the contrast of LCD1602
- RS: A register select pin that controls where in the LCD's memory you are writing data to. You can select either the data register, which holds what goes on the screen, or an instruction register, which is where the LCD's controller looks for instructions on what to do next.
- R/W: A Read/Write pin that selects reading mode or writing mode
- E: An enabling pin that, when supplied with low-level energy, causes the LDC module to execute relevant instructions.
- D0-D7 : Pins that read and write data A and K: Pins that control the LED backlight

We connected the display to arduino as follows:



- GND and VCC to power supply of breadboard
- VO to potentiometer
- Rs to digital pin (A7) of arduino
- Rw to ground of breadboard
- E to digital pin (A8) of arduino
- D4 to digital pin (A6) //
- D5 to digital pin (A5) //
- D6 to digital pin (A4) //
- D7 to digital pin (A3) //
- A to 5V of the breadboard
- K to ground of breadboard

Fig. 6: LCD display 1602

We used a potentiometer up 10kΩ to adjust the display contrast.

```
if { BPM ==0;

    lcd.setCursor(0,0);
    lcd.print("THE SENSOR IS    ");
    lcd.setCursor(0,1);
    lcd.print("NOT WORKING    ");
}
else {

    lcd.setCursor(0,0);
    lcd.print("SENSOR= KY-039");
    lcd.setCursor(0,1);
    lcd.print("HR= ");
    lcd.print(BPM);
    lcd.print(" BPM");
}
}
```

Real circuit

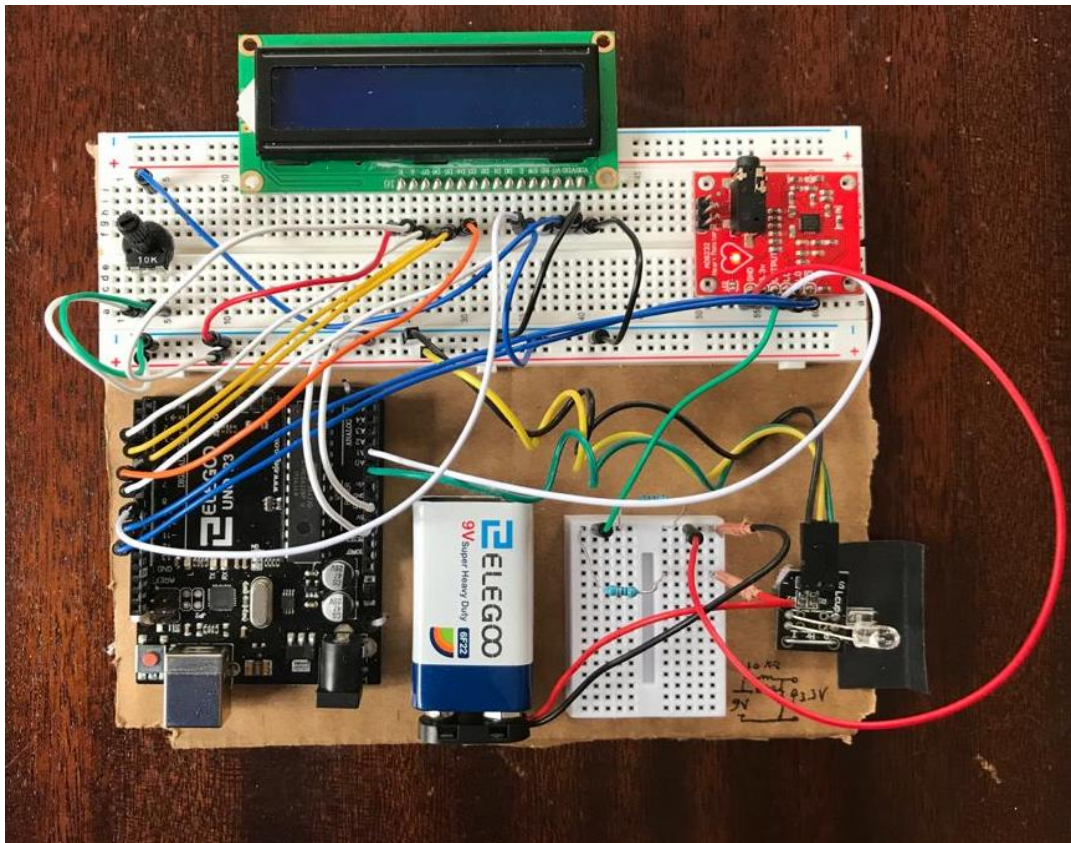


Fig. 7: real circuit made by ourselves

2. Communication

We used the COM3 in order to obtain the information from arduino. COM3 is a port of communications in the computer that is used to transmit data from hardware devices to the computer processor. Even though most computers use only COM1 and COM2 it may be necessary to add the COM3 when there is another additional machines using the port of communications, in this case the Arduino.

3. Software development

3.1 Data acquisition

For the data acquisition we used a software called Tera Term that allow us to upload the data from the sensors in String, into a csv file. The file has the time, values from ky-039 and from the ecg, allowing us to compare them later on Matlab.

The arduino as a function called millis(), that gives the current time, the “media” is the data from ky-039 filtered (that we are going to discuss later) and the “analogRead(A1)” is the data from the ecg.

```
Serial.print(millis());
Serial.print(";"+String(media)+";");
if((digitalRead(10) == 1) || (digitalRead(11) == 1)){
    Serial.println('!');
}
else{
    // send the value of analog input 0:
    Serial.println(analogRead(A1));
}
```

reference: <https://tssh2.osdn.jp/index.html.en>

Extra:

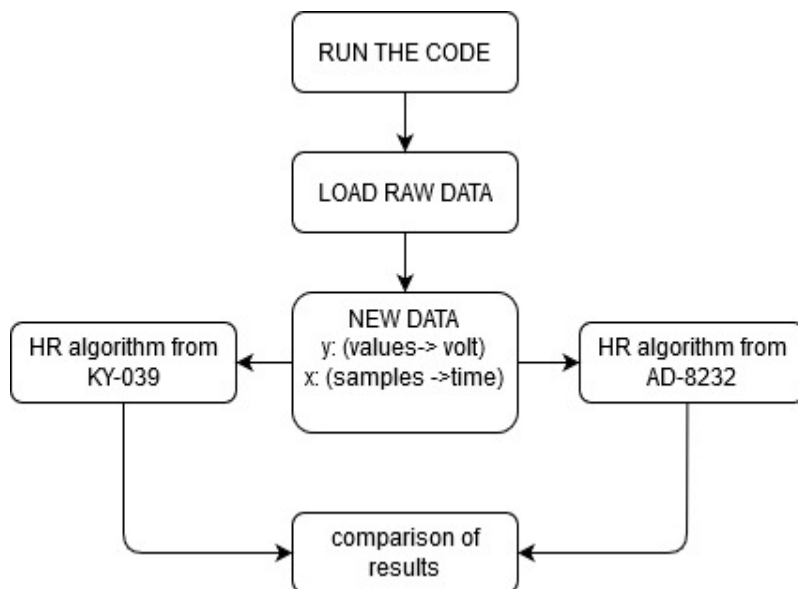
We also explored how to record the data straight into matlab. It's possible to download a package of arduino for matlab which allows to identify arduino as an object in this way: 'a=arduino('COM3','uno')'. Where 'COM3' is the serial port we used to transfer the data and 'uno' is model of arduino.

Despite the practicality of this method we have abandoned it because it shows some weaknesses on the acquisition speed working with a much lower sampling rate than the arduino one.

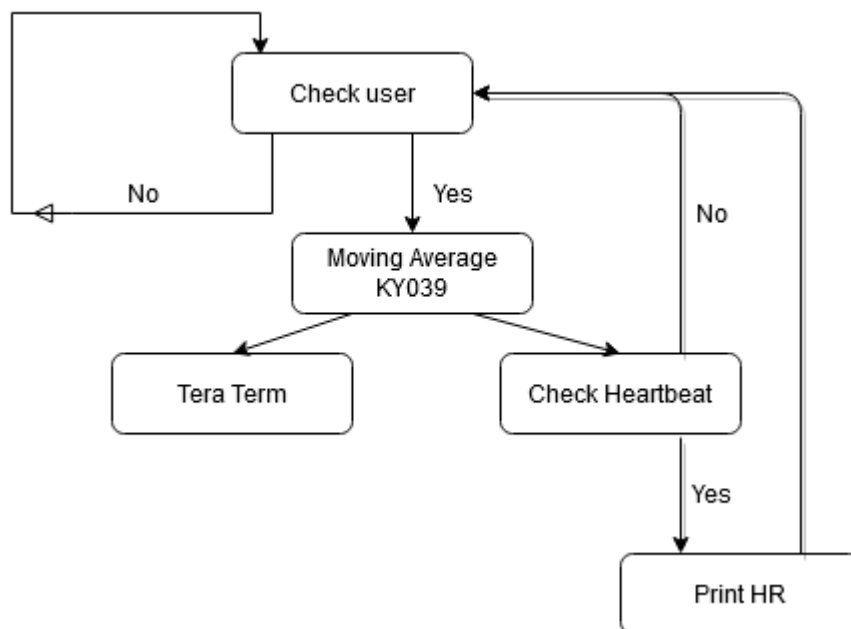
reference: <https://www.mathworks.com/help/supportpkg/arduinoio/>

3.2 Flowchart

3.2.1 Flowchart Matlab



3.2.2 Flowchart arduino



3.3 Coding and debugging

We divide this in two part:

1. Code in arduino to show real time Heart Rate
2. Code in matlab to further study and analyze the sensor

3.3.1 Data processing: Arduino

First we have to understand if there is a finger in the sensor or not. for this we created a threshold. When there isn't a finger, there is more infrared light reaching the sensor, therefore, more resistance that implies less voltage. Values that are below the threshold indicates inexistence of a finger and values above means that there is a finger between the sensor and the LED. We set the values to zero in case of absence of finger, because, when the user is changed the sensor needs to calibrate again

```
int threshold=500; // to see if there is or not the presence of a finger.

//Wait for a bit to keep serial data from saturating
raw=analogRead(A0);
// coditions to check if there is a finger or not
if(raw < threshold){
    raw=0;
    nmax=0;
    delay(1000);
    Serial.println("No data");
    a=0;
    timer=millis(); // start counting time
}
```

When a finger is present, we start collecting data and processing it. We start to apply a moving average of the last 40 values of the raw data from the sensor and saving it in an array of 100 values. This allow us to remove repeated values (since data acquisition is fast) and to reduce the noise, since the moving average is a good FIR filter to attenuate random noise.

Compute the average of M points:

$$y[i] = \frac{1}{M} \sum_{j=0}^{M-1} x[i+j]$$



```

val[39]=raw;

for (int i=0;i<39;i++){
    val[i]=val[i+1];
}

media = 0;
for (int i=0;i<40;i++){
    media=media+val[i];
}
media=media/40;

t[99]=media;

for (int i=0;i<99;i++){
    t[i]=t[i+1];
}

```

We then applied a condition that allow us to check if there is an heartbeat. Basically, if the middle value of the Moving Average array is the larger relatively to the right or left values, it means that there is an heartbeat.

```

pek=1;
mai=t[50];
for (int i=0; i<100;i++){
    if (t[i]>mai || t[0]==0 ){
        pek=0;
    }
}

```

We then count the number of heartbeats and we keep on dividing the value by the interval of time, that is obtained with the function `millis()`. To complement the condition above explained, we created another condition that avoid false positives after 300ms (this only silence other true positives if the heart rate is higher than 200BPM) of the first true positive.

For more accurate values, the heartbeat only shows after 5 consecutive heartbeats and resets after 7 consecutive. This helps the algorithm to not use very old values and to stay updated. For the BPM we also did a moving average of the previous BPMs.

```

if(pek==1){ // avoid second heartbeats that come before 300ms
  if(millis()-start>300){
    nmax++;
    if(a==0 || nmax>2){ // nmax>2 because its avoid error in the hbeat calculations
      hbeat=(nmax*30/((millis()-timer)/1000)*2);
      hbeatma[2]=hbeat;
      for (int i=0;i<2;i++){
        hbeatma[i]=hbeatma[i+1];
      }
    }
    if(nmax<5 && a==0){
      Serial.println("Collecting data");
    }
    else {
      BPM=0;
      for (int i=0;i<3;i++){
        BPM=BPM+hbeatma[i];
      }
      Serial.println("BPM: "+String(BPM/3));
    }
    start= millis(); // after detecting a heartbeat the "start" resets for the first condition
  }
}

```

This code is inside of the void loop() that is repeated where the repeat rate is related to the sampling frequency. The declaration of the variables are inside the void setup() that only run one time and is the first thing to run.

3.3.2 Data processing: Matlab

The processing of the data in matlab can be divided in 3 parts:

1. Load data from arduino
2. Use of the algorithms to calculate the heart rate for both the sensors
2. Plotting of results and statistical analyst

After recording the data via arduino we turned them into 'file.mat'. The data from 'KY-039' have already been filtered with arduino. First of all we cut the data taking only 1000 samples, we did it to show more visibly the features of the signals in the figure and as a matter of practicality.

We note that taking only 1000 samples couldn't be enough to compare the sensors statistically, so we recommend to who want to explore this point to analyze a wide time interval. After this we calculated the sampling frequency of the signal dividing the number of samples by the acquisition time interval (both information are contained in the loaded date columns 'data.mat').

We converted the data into volt knowing that arduino record the analog input in a range from 0 to 1023 while the real value is from 0 to 5V: so we multiplied each value for 5 and divided by 1023.

```

fs_ecg=53.47; %sampling frequency ecg
samples=0:1/fs_ecg:20; %to pass from samples to sec
samples=samples(1:1000);
pulse=pulse(1:1000);
pulse=pulse.*(5/1023); %to pass in volts

```

Finally we plot the signals setting a limit to y-label for a better view.

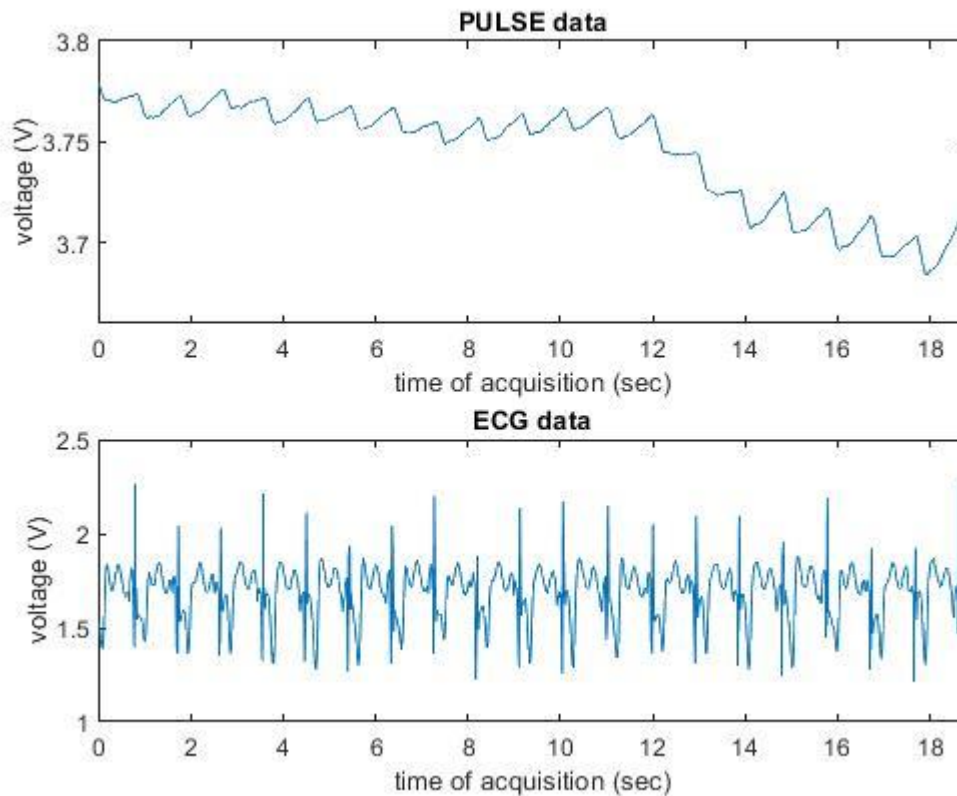


Fig. 8: pulse data are from 'KY-039', ecg data are from 'AD-8232'

To calculate the heart rate we used two different criteria:

1. Pulse algorithm:

for the pulse data we used the 'ginput' function to save in a vector all the positions of the peaks.

Knowing the positions of the peaks (previous converted into seconds) we can calculate the heart rate in beat for minute divided 60 sec by the peak time difference.

```

%% Algorithm to calculate the heart rate from pulse

for a=1:21
    fhr_pulse(a)=60./(x(a+1)-x(a));
end

fhr_pulse(:,15) = [ ]; %we deleted the 15' and 21' columns beacause there are Infinities
fhr_pulse(:,20) = [ ];

```

2. ECG algorithm:

This algorithm is a bit more complicated of previous one because of the possibility to use it without counting manually the positions of the peaks. However It's works on the choice of threshold above which we analyze the slope of the signal in search of the absolute maximum (peak). The description of each step has been extensively commented in the code in case you wish to explore it.

```

%% Algorithm to calculate the heart rate from ecg
pause()
hold on
k1=find(ecg_1>1.9); %k is a vector where its elemetns are the index of the elements of Y1 which are greater than 1.9
k2=diff(k1); %k2 is the difference vector of K1, it's= K1-1
kf=find(k2>10); %kf is a vector whose elements are the positions of K2 values greater than 20
ki=[k1(kf+1)]; %k1(1) contains the position of the first value above threshold
plot(samples(ki),ecg_1(ki),'r*')
legend('ecg','spikes')

nc=fs_ecg*0.03; % where 0.03 sec is the duration interval of an R wave, nc=num samples*duration
m=zeros(length(ki),1);
for i=1:length(ki)
    [y,j]=max(ecg_1(ki(i):ki(i)+nc-1));
    m(i,1)=j+ki(i)-1;
end

rrc=diff(m); % distance in samples between peaks R in this way we obtained the R-R differences in samples.
% Then we converted them into seconds
rrs=rrc*0.004;

fhr_ecg=60./rrs; %%60= 1 min --> beats per minute (BPM)
n_spikes=length(m)-1;
fhr_ecg(1:n_spikes);
rrc=diff(m);
rrs=rrc/fs_ecg;
fhr_ecg=60./rrs;
fhr_ecg(1:n_spikes);

```

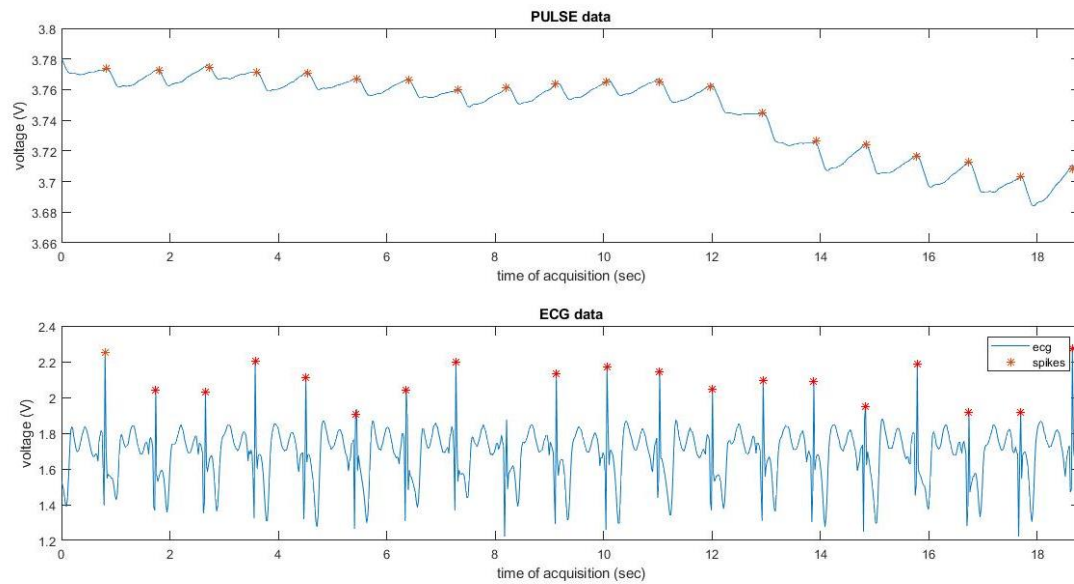


Fig. 9: plot pulse and ecg signals

From the figure above we note how the signals have the same number of peaks and how these coincide temporally, this makes us think a good accuracy in sampling signals at the same time and thus the possibility of being able to compare them and obtain reliable results.

Finally we plot heart rate , square mean and variance for both sensors.

```
plot(fhr_ecg, 'o')
hold on
plot(fhr_pulse, '*')
title('HEART BEAT (beats per minute)');
xlabel('num. beats examined');
ylabel('val. beats examined');
legend('ecg HR', 'pulse HR')
```

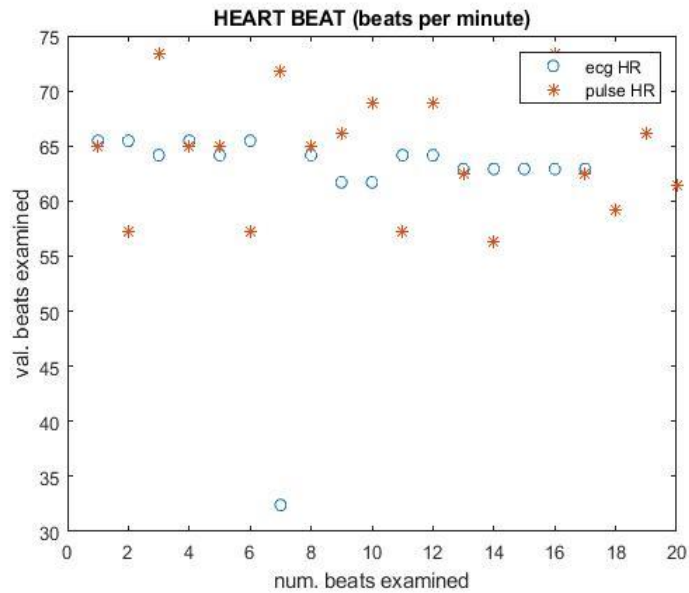


Fig. 10: heart rate calculated by ecg and pulse signals

```

rms(fhr_pulse)
rms(fhr_ecg)

fhr_pulse=fhr_pulse';

fhr_pulse=fhr_pulse-mean(fhr_pulse);
var(fhr_pulse)

fhr_ecg=fhr_ecg-mean(fhr_ecg);
var(fhr_ecg)

```

	KY-039 (PULSE)	AD-8232 (ECG)
HR square mean	64.33	61.94
HR variance	29.37	1.64

tab. 1: square mean and standard deviation of heart rate gauged with ky-039 and ad-8232 in a temporal window of 18 seconds

We noted how big is the different in the variance between the sensors.
It clearly shows how to calculate the heartbeat with ecg is much more efficient.

4. Demo

3.1 [Demo voltage drop](#)

This video shows how the kit AD-8232 works thanks to the DC-voltage from the battery. It's possible to see the blinking of the led when we link the battery to the kit.

3.2 [Demo sensor KY-039 and display contrast](#)

This video shows how the sensor KY-039 works plotting in real time the Heart Rate on the LCD display. It's also shown how It's possible to change the contrast of the LCD display by changing the resistance of the potentiometer.

4. Conclusions

The ECG outperforms the KY039 in terms of accuracy but the KY039 is more viable since there are less requirements to get the data.

The results were expected since the noise in the ECG is very reduced compared to the KY039, nonetheless, the results for the KY039 were better than we expected. The arduino code detects very well the heartbeat but the heart rate calculations could be improved. Using the matlab we could study and get good conclusions about the KY039.